

In this chapter, we'll create and manipulate lists using list comprehensions, which you'll probably like after you get used to them. The following is a very simple list comprehension that creates a copy of our `word_list`:

---

```
word_list_copy = [word for word in word_list]
```

---

You can see that the syntax `for word in word_list` is very similar to the syntax for a `for` loop. But we don't need a colon or extra lines. In this case, the list comprehension is as simple as possible, just specifying that we want each word in `word_list` to be in our new list, `word_list_copy`. This may not be so useful, but we can concisely add logic to make it more useful. For example, if we want to find every word in our word list that contains the letter *n*, all it takes is the simple addition of an `if` statement:

---

```
has_n = [word for word in word_list if 'n' in word]
```

---

We can run `print(has_n)` to see that the result is what we expect:

---

```
['one', 'divine']
```

---

Later in the chapter, you'll see more complex list comprehensions, including some that have nested loops. However, all of them follow the same basic pattern: a `for` loop specifying iteration, with optional `if` statements describing the logic of what we want to select for our final list output.

We'll use Python's `re` module to access text manipulation tools. One of `re`'s useful functions is `finditer()`, which can search our text to find the location of any word in our `word_list`. We use `finditer()` in a list comprehension like so:

---

```
import re
locs = list(set([(m.start(),m.end()) for word in word_list for m in
re.finditer(word, text)]))
```

---

That line is a little dense, so take a moment to make sure you understand it. We're defining a variable called `locs`, short for "locations"; this variable will contain the locations in the text of every word in our word list. We'll use a list comprehension to get this list of locations.